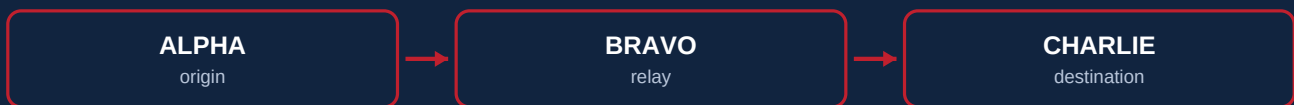


GREYNOC FIELD MANUAL

Creating a Simple P2P PGP Node Transfer Workflow

Termux, local Wi-Fi, Bluetooth fallback, SSH/SCP, and store-and-forward relays



ciphertext only · forwarded unchanged

Transport can be Wi-Fi, hotspot, Bluetooth share, USB, or removable media. Encryption is independent of transport.

Document ID	GN-PUB-PGPNODE-001
Version	1.2 (Public Training Edition)
Release date	2026-06-17
Classification	Public · unrestricted training material

Use statement

This guide is for lawful, authorized, privacy-focused training, emergency readiness, and local encrypted file transfer between devices you own or are permitted to use. It is not a guide for unauthorized access, evasion, data theft, or misuse.

Version 1.2 release note

This release builds on the v1.1 wording and text-layer repairs. It adds a consolidated Field Tips and Tricks section (QR key exchange, detached signatures, symmetric fallback, multi-recipient encryption, archive bundling, rsync, machine-readable signature checks, and air-gap notes), a new Automation Scripts section providing copy-paste Bash helpers for setup, encryption, decryption, relay, key-QR export, and a self-stopping pull server, plus a formal document-control block and version history. The QA checklist and source notes were refreshed against public documentation current on 2026-06-17.

Document control

Field	Value
Document ID	GN-PUB-PGPNODE-001
Title	P2P PGP Node Transfer Workflow — Public Training Edition
Template	GN-TPL-PUB-001 (public guide, navy/red)
Owner	GreyNOC Operations
Audience	Public · operators, students, privacy-focused readers
Status	Released
Current version	v1.2
Release date	2026-06-17
Review cycle	Per release; re-verify external sources each bump

Version history

Version	Date	Summary of change
1.0	2026	Initial public training release of the node-transfer workflow.
1.1	2026-06-17	Tightened public wording, repaired the text layer, removed cramped internal QA language, clarified Termux:API, Wi-Fi Direct, GnuPG trust, and store-and-forward limits.
1.2	2026-06-17	Added Field Tips and Tricks and Automation Scripts sections; added document control and version-history blocks; refreshed QA checklist and source notes.

Contents

- | | |
|-----------------------------------|------------------------------------|
| 1. Purpose and Scope | 11. Store-and-Forward Relay Method |
| 2. Safety, Legal, and Trust Model | 12. Naming, Hashing, and Manifests |

- | | |
|--|---|
| 3. Architecture Overview | 13. Security Hygiene and Lost Device Response |
| 4. Required Tools | 14. Two-Phone Field Test |
| 5. Node Folder Layout | 15. Field Tips and Tricks |
| 6. PGP Identity Setup | 16. Automation Scripts |
| 7. Public Key Exchange and Trust | 17. Limits and Design Notes |
| 8. Encrypting, Signing, and Decrypting | 18. Release QA Checklist |
| 9. Local Transport Modes | 19. Source Notes |
| 10. SSH and SCP Transfers | |

1. Purpose and Scope

This manual explains a simple encrypted node-transfer workflow using Android devices, Termux, a shared local network or hotspot, optional Bluetooth file sharing, SSH/SCP, and OpenPGP through GnuPG.

The goal is not to replace mature mesh or emergency-communications tools. The goal is to give operators a low-cost, inspectable pattern where files are encrypted before transport and can be relayed as ciphertext.

2. Safety, Legal, and Trust Model

Use this workflow only with devices, networks, keys, and accounts you own or have explicit permission to operate. Do not use it to access unauthorized systems, hide criminal activity, move stolen data, impersonate users, or bypass lawful controls.

The trust model assumes that local networks, relays, Bluetooth handoff, temporary file servers, removable media, and nearby observers may be untrusted. OpenPGP protects message content; it does not hide metadata such as file names, timing, device proximity, or participants.

Core rule

Encrypt first. Transfer second. Verify the key, hash, and signature before trusting the decrypted content. Relays should handle only ciphertext.

3. Architecture Overview

Each Android device acts as a node. A node may originate a file, relay a file, or receive and decrypt a file. Relay nodes do not need the plaintext because the origin encrypts the file to the destination public key.



Relays handle encrypted files only; the destination verifies and decrypts.

Figure 1. Encryption-first workflow for local transfer and manual relay.

For training, use a local-only path: a phone hotspot, shared Wi-Fi network, USB/SD movement, or Android Bluetooth sharing for small encrypted files. Internet access is not required for the local transfer itself.

4. Required Tools

Install Termux from F-Droid or the official GitHub release channel. Avoid old Play Store builds. Termux provides the command environment and package manager; Android still controls platform features such as Wi-Fi Direct and Bluetooth pairing.

```
pkg update
pkg upgrade
pkg install gnupg openssh python rsync nano qrencode coreutils
# Optional: install only if you also install the Termux:API add-on app
pkg install termux-api
gpg --version
ssh -V
python --version
```

Termux:API is optional. Its helper package works with the separate Termux:API Android add-on app, and both apps must come from a compatible signing source. It does not turn stock Android into a full Bluetooth or Wi-Fi Direct controller.

5. Node Folder Layout

```
mkdir -p ~/greynoc-node/{inbox,outbox,relay,keys,logs,tmp}
chmod 700 ~/greynoc-node
chmod 700 ~/greynoc-node/*
```

Folder	Purpose
inbox	received encrypted files
outbox	encrypted files ready to send
relay	ciphertext waiting to be forwarded
keys	public keys, fingerprints, and revocation records
logs	manual transfer notes and manifests
tmp	temporary working area (wipe after use)

6. PGP Identity Setup

Create one OpenPGP identity per node or operator. Use clear node names, protect the private key with a strong passphrase, and set training keys to expire so stale lab keys age out.

```
gpg --full-generate-key
gpg --list-keys
gpg --list-secret-keys
gpg --armor --export alpha@greynoc.local \
  > ~/greynoc-node/keys/alpha-public.asc
# Create and store a revocation certificate offline
gpg --output revoke-alpha.asc --gen-revoke alpha@greynoc.local
```

Keep private keys and revocation certificates out of relay folders. Do not upload private keys to temporary HTTP servers or shared storage.

7. Public Key Exchange and Trust

Import another node public key, then verify the fingerprint through a trusted out-of-band channel such as in-person comparison, voice confirmation, or a QR code. Do not trust a public key only because it arrived over the same network path as the file.

```
gpg --import ~/greynoc-node/keys/bravo-public.asc
gpg --fingerprint bravo@greynoc.local
# After out-of-band fingerprint verification, local-sign the key
gpg --lsign-key bravo@greynoc.local
```

Trust wording

In GnuPG, the edit-key command named *trust* changes ownertrust: how much you trust that person to certify other keys. It is not the same thing as verifying a fingerprint. For simple field training, verify fingerprints and local-sign known peer keys; change ownertrust only when you understand the Web of Trust impact.

8. Encrypting, Signing, and Decrypting Files

Encrypt text or binary files before transfer. ASCII-armored files are easier to inspect and copy; binary .pgp files are smaller and better for larger payloads. The recipient must have the sender public key available if they need to verify the signature.

```
echo "GreyNOC field test message" > message.txt
gpg --armor --sign --encrypt \
  --local-user alpha@greynoc.local \
  --recipient bravo@greynoc.local \
  --output message.txt.asc message.txt
gpg --output decrypted-message.txt --decrypt message.txt.asc
```

When decrypting a signed encrypted file, read the GnuPG status output. A successful decrypt is not enough by itself; confirm that the signature matches the expected sender key.

9. Local Transport Modes

Mode	Best use	Limits
Same Wi-Fi or hotspot	Normal local file transfer by SSH/SCP or HTTP pull	Requires all active nodes to join the same network
Bluetooth sharing	Fallback for public keys, fingerprints, or small encrypted files	Android behavior varies by version, device, OEM, and enabled profiles
Wi-Fi Direct	App/API-driven peer discovery and connection	Stock Termux alone does not provide an automatic Wi-Fi Direct mesh
USB/SD/removable media	Offline handoff of ciphertext	Track media custody and verify hashes
Temporary HTTP pull	Quick one-to-many pull of encrypted outbox files	Exposes filenames and timing; stop the server after use

The simplest practical mode is a phone hotspot or shared Wi-Fi network. One device starts the hotspot, other nodes join, and encrypted files move over SSH/SCP or a temporary pull server. Validate Bluetooth and Wi-Fi Direct behavior on the exact Android versions and devices before field use.

10. SSH and SCP Transfers

On the receiving node, start the SSH server and confirm the Termux username and IP address. Termux SSH commonly listens on port 8022. Use SSH keys for repeat workflows; password authentication is acceptable only for controlled training if you rotate it afterward.

```
# Receiver
sshd
whoami
passwd
ip -brief addr
ss -tulpn | grep ssh

# Sender: transfer only encrypted files
scp -P 8022 message.txt.asc \
u0_a123@192.168.43.101:~/greynoc-node/inbox/
```

For a temporary pull server, host only encrypted output files, avoid sensitive filenames, and stop the server immediately after transfer.

```
cd ~/greynoc-node/outbox
python -m http.server 8080
# Receiver
wget http://192.168.43.100:8080/message.txt.asc \
-O ~/greynoc-node/inbox/message.txt.asc
```

11. Store-and-Forward Relay Method

The relay method lets a node forward ciphertext without decrypting it. Alpha can encrypt to Charlie, hand the file to Bravo, and Bravo can forward it to Charlie unchanged. This is useful when all nodes cannot see each other at the same time.

```
gpg --armor --sign --encrypt \
--local-user alpha@greynoc.local \
--recipient charlie@greynoc.local \
--output alpha-to-charlie.asc message.txt
scp -P 8022 alpha-to-charlie.asc \
bravo@192.168.43.101:~/greynoc-node/relay/
# Relay node forwards unchanged
scp -P 8022 ~/greynoc-node/relay/alpha-to-charlie.asc \
charlie@192.168.43.105:~/greynoc-node/inbox/
```

If the relay needs proof of custody, use a relay manifest and hashes. Do not decrypt-and-reencrypt at the relay unless the operating plan explicitly requires the relay to become a trusted endpoint.

12. Naming, Hashing, and Manifests

Use predictable file names and hashes so relays and recipients can track files without opening them. Avoid sensitive details in filenames when observers may see directory listings or HTTP requests.

```
YYYYMMDD-HHMM-FROM-TO-TYPE-ID.asc
20260617-1430-alpha-charlie-sitrep-001.asc

sha256sum message.txt.asc > message.txt.asc.sha256
sha256sum -c message.txt.asc.sha256
```

Relay manifest field	Purpose
File	Exact encrypted filename
Origin / destination / relay node	Routing and custody tracking

Relay manifest field	Purpose
Date/time and transport	Operational timeline
Encrypted / signed	Expected protection state
SHA256 hash and notes	Integrity check and exception notes

13. Security Hygiene and Lost Device Response

- Use strong Android lock settings and strong OpenPGP passphrases.
- Do not share private keys or store passphrases in notes apps.
- Stop SSH and temporary HTTP servers when finished.
- Rotate hotspot passwords after training or field use.
- Revoke and replace keys if a device is lost, seized, or shared outside the trust boundary.
- Keep revocation certificates offline and away from relay folders.

```
gpg --output revoke-alpha.asc --gen-revoke alpha@greynoc.local
```

14. Two-Phone Field Test

Minimum validation uses two Android phones on the same hotspot or Wi-Fi network. Each phone installs tools, creates a key, exchanges public keys, verifies fingerprints, and transfers one encrypted test file.

```
# Phone A
gpg --armor --export alpha@greynoc.local > alpha-public.asc
# Phone B
gpg --armor --export bravo@greynoc.local > bravo-public.asc

# After verified key exchange, Phone A encrypts to Phone B
echo "Test from Alpha to Bravo" > test.txt
gpg --armor --sign --encrypt \
  --local-user alpha@greynoc.local \
  --recipient bravo@greynoc.local \
  --output test-alpha-bravo.asc test.txt
scp -P 8022 test-alpha-bravo.asc \
  USERNAME@PHONE_B_IP:~/greynoc-node/inbox/

# Phone B decrypts and reviews signature status
gpg --decrypt ~/greynoc-node/inbox/test-alpha-bravo.asc
```

15. Field Tips and Tricks

These techniques close common gaps that show up during field use. They use only tools already installed in Section 4 and stay inside the same encrypt-first, verify-before-trust model.

15.1 QR code key exchange

An armored public key is plain text, so it can be rendered as one or more QR codes and scanned out-of-band. This avoids sending keys over the same path as files and gives a clean in-person fingerprint check. Keep QR exchange to public keys only.


```
# Show the key as a QR block in the terminal
gpg --armor --export alpha@greynoc.local | qrencode -t ANSIUTF8

# Or write a PNG to hand off / display
gpg --armor --export alpha@greynoc.local \
  | qrencode -o ~/greynoc-node/keys/alpha-public-qr.png

# Always confirm the fingerprint verbally after import
gpg --fingerprint alpha@greynoc.local
```

15.2 Detached signatures and manifest signing

A detached signature lets you prove authorship of a file or a whole manifest without changing the file itself. This is ideal for signing a relay manifest or a checksum list so recipients can confirm both integrity and origin.

```
# Sign a checksum manifest (origin)
sha256sum *.asc > manifest.sha256
gpg --local-user alpha@greynoc.local \
  --output manifest.sha256.sig --detach-sign manifest.sha256

# Verify the manifest, then the files (recipient)
gpg --verify manifest.sha256.sig manifest.sha256
sha256sum -c manifest.sha256
```

15.3 Symmetric fallback when no public key is available

If a peer has no usable public key, you can still protect a file with a passphrase-based symmetric cipher and share the passphrase over a separate trusted channel. This trades the key-verification guarantees of public-key mode for simplicity, so reserve it for low-sensitivity training transfers.

```
# Encrypt with a passphrase (AES-256)
gpg --symmetric --cipher-algo AES256 \
  --output message.txt.gpg message.txt

# Decrypt (prompts for the same passphrase)
gpg --output message.txt --decrypt message.txt.gpg
```

15.4 Encrypt to several recipients at once

For one-to-many distribution, list multiple recipients. Every named recipient (and the optional sender, via encrypt-to-self) can decrypt the single file.

```
gpg --armor --sign --encrypt \
  --local-user alpha@greynoc.local \
  --recipient bravo@greynoc.local \
  --recipient charlie@greynoc.local \
  --recipient alpha@greynoc.local \
  --output sitrep-multi.asc sitrep.txt
```

15.5 Bundle folders before encrypting

GnuPG encrypts a single stream, so pack a directory into one archive first. tar preserves structure and timestamps; the result is encrypted as one file.

```
tar -czf payload.tar.gz ./report-dir
gpg --sign --encrypt \
  --local-user alpha@greynoc.local \
  --recipient charlie@greynoc.local \
  --output payload.tar.gz.gpg payload.tar.gz
shred -u payload.tar.gz # remove the plaintext archive
```

15.6 Larger or repeated sets over SSH with rsync

For many files or resumed transfers, rsync over the Termux SSH port is more efficient than repeated scp. Move ciphertext only.

```
rsync -avz -e 'ssh -p 8022' \
  ~/greynoc-node/outbox/ \
  u0_a123@192.168.43.101:~/greynoc-node/inbox/
```

15.7 Machine-readable signature verification

Reading human output is fine by hand, but scripts should check the status stream. A trustworthy result shows both a VALIDSIG line and a successful decryption, so automation keys on those tokens rather than the exit code alone.

```
gpg --status-fd 1 --decrypt message.txt.asc \
  > /dev/null 2> /tmp/gpg.err
grep -q 'VALIDSIG' /tmp/gpg.err && echo 'signature OK'
```

15.8 Keep clocks honest

Signature and key-expiry checks depend on the device clock. On phones with no network, confirm date and time before a field test so a valid signature is not rejected as future-dated or expired.

```
date # confirm before generating or verifying signatures
```

15.9 Air-gap and removable-media handoff

Because encryption is independent of transport, the same .asc file can cross an air gap on an SD card or USB drive. Hash on write, hash on read, and record media custody in the manifest. Treat any media that left your control as untrusted until re-verified.

16. Automation Scripts

These Bash helpers wrap the manual commands above so routine transfers are consistent and auditable. Save them under ~/greynoc-node/bin, make them executable, and read each one before use. They never handle private keys or passphrases on the command line, never decrypt at a relay, and write a manifest line for every action. Adjust the node names, IPs, and port to your own lab.

```
mkdir -p ~/greynoc-node/bin
# paste each script below, then:
chmod 700 ~/greynoc-node/bin/*.sh
```

16.1 gn-setup.sh — prepare folders and verify tools

```
#!/data/data/com.termux/files/usr/bin/bash
# gn-setup.sh - create the node layout and confirm required tools
set -euo pipefail
ROOT="${GN_ROOT:-$HOME/greynoc-node}"
mkdir -p "$ROOT"/{inbox,outbox,relay,keys,logs,tmp,bin}
chmod 700 "$ROOT" "$ROOT"/*
for t in gpg ssh scp rsync sha256sum qrencode; do
    command -v "$t" >/dev/null 2>&1 || echo "MISSING: $t"
done
echo "node root ready: $ROOT"
```

16.2 gn-encrypt.sh — encrypt, sign, hash, log

```
#!/data/data/com.termux/files/usr/bin/bash
# gn-encrypt.sh <sender> <recipient> <type> <id> <plaintext-file>
# Produces an armored, signed, encrypted .asc in outbox plus a sha256 and log line.
set -euo pipefail
ROOT="${GN_ROOT:-$HOME/greynoc-node}"
SENDER="$1"; RCPT="$2"; TYPE="$3"; ID="$4"; SRC="$5"
STAMP="$(date +%Y%m%d-%H%M)"
OUT="$ROOT/outbox/${STAMP}-${SENDER%%@*}-${RCPT%%@*}-${TYPE}-${ID}.asc"
gpg --armor --sign --encrypt \
    --local-user "$SENDER" --recipient "$RCPT" \
    --output "$OUT" "$SRC"
sha256sum "$OUT" | tee "$OUT.sha256"
printf '%s\tencrypt\t%s\t->%s\t%s\n' \
    "$(date -Iseconds)" "$(basename "$OUT")" "$RCPT" \
    "$(cut -d' ' -f1 "$OUT.sha256")" >> "$ROOT/logs/manifest.tsv"
echo "wrote $OUT"
```

16.3 gn-decrypt.sh — verify hash and signature, then decrypt

```
#!/data/data/com.termux/files/usr/bin/bash
# gn-decrypt.sh <ciphertext.asc>
# Confirms the sha256 (if present) and a valid signature before decrypting.
set -euo pipefail
ROOT="${GN_ROOT:-$HOME/greynoc-node}"
CT="$1"
[ -f "$CT.sha256" ] && sha256sum -c "$CT.sha256"
if gpg --status-fd 1 --decrypt "$CT" 2>/tmp/gn.err \
    >"$ROOT/tmp/${basename "${CT%.asc}").out"; then
    if grep -q 'VALIDSIG' /tmp/gn.err; then
        echo "signature OK -> $ROOT/tmp/${basename "${CT%.asc}").out"
    else
        echo "WARNING: decrypted but signature NOT validated" >&2
    fi
else
    echo "decrypt failed" >&2; exit 1
fi
printf '%s\tdecrypt\t%s\n' "$(date -Iseconds)" \
    "$(basename "$CT")" >> "$ROOT/logs/manifest.tsv"
```

16.4 gn-relay.sh — forward ciphertext unchanged with custody log

```
#!/data/data/com.termux/files/usr/bin/bash
# gn-relay.sh <ciphertext.asc> <user@host>
# Forwards a file from the relay folder WITHOUT decrypting it.
set -euo pipefail
ROOT="${GN_ROOT:-$HOME/greynoc-node}"
PORT="${GN_SSH_PORT:-8022}"
CT="$1"; DEST="$2"
BEFORE="$(sha256sum "$CT" | cut -d" " -f1)"
scp -P "$PORT" "$CT" "$DEST:~/greynoc-node/inbox/"
printf '%s\trelay\t%s\t->%s\t%s\n' "$(date -Iseconds)" \
    "$(basename "$CT")" "$DEST" "$BEFORE" >> "$ROOT/logs/manifest.tsv"
echo "relayed $(basename "$CT") (hash $BEFORE)"
```

16.5 gn-keyqr.sh — export a public key and render a QR

```
#!/data/data/com.termux/files/usr/bin/bash
# gn-keyqr.sh <key-id-or-email>
# Exports the public key, prints a QR to the terminal, and saves a PNG.
set -euo pipefail
ROOT="${GN_ROOT:-$HOME/greynoc-node}"
KEY="$1"
ASC="$ROOT/keys/${KEY%*}-public.asc"
gpg --armor --export "$KEY" > "$ASC"
gpg --fingerprint "$KEY"
qrencode -t ANSIUTF8 < "$ASC"
qrencode -o "$ROOT/keys/${KEY%*}-public-qr.png" < "$ASC"
echo "public key + QR written for $KEY"
```

16.6 gn-pull.sh — self-stopping temporary pull server

```
#!/data/data/com.termux/files/usr/bin/bash
# gn-pull.sh [seconds]
# Serves ONLY the outbox over HTTP, then stops automatically (default 120s).
set -euo pipefail
ROOT="${GN_ROOT:-$HOME/greynoc-node}"
TTL="${1:-120}"
cd "$ROOT/outbox"
python -m http.server 8080 &
PID=$!
echo "serving outbox on :8080 for ${TTL}s (pid $PID)"
sleep "$TTL"
kill "$PID" 2>/dev/null || true
echo "pull server stopped"
```

Script trust note

Review every script before running it, keep them in a 700 folder, and never feed a relay script anything but ciphertext. The decrypt helper warns loudly when a file decrypts but its signature does not validate — treat that warning as a failed transfer, not a minor note.

17. Limits and Design Notes

- This is a simple encrypted node-transfer workflow, not a full automatic routing mesh.
- Termux alone should not be treated as a complete Bluetooth networking controller on stock Android.

- Android Wi-Fi Direct is app/API-driven and device behavior varies by Android version and manufacturer.
- Manual relay is practical because ciphertext can be forwarded over any authorized path.
- OpenPGP protects content, but filenames, timing, device proximity, and transfer participants may still be visible.
- Symmetric (passphrase) mode is a convenience fallback and gives up public-key fingerprint verification; prefer signed public-key mode.
- A relay that renames, decrypts, or reencrypts files becomes part of the trust boundary and must be handled as a trusted endpoint.

18. Release QA Checklist

Check	Result	Notes
Safety framing	Pass	Lawful-use scope and misuse boundaries remain prominent on the cover and in Sections 2 and 17.
Technical accuracy	Pass	Termux, Termux:API, Android Wi-Fi Direct, GnuPG, qrencode, and rsync wording checked against official sources.
New content review	Pass	Tips/tricks and scripts use only Section 4 tools; scripts log to a manifest and never decrypt at a relay.
Text layer	Pass	No mojibake; en-dash bullets and code blocks extract cleanly.
Layout render	Pass	Cover, tables, diagrams, and dark code blocks checked for clipping, bleed, and overlap.
Release labels	Pass	Cover, headers, footers, document control, version history, and filename bumped to v1.2.

19. Source Notes

Release QA used public documentation available on 2026-06-17. The sources support the design choice to treat Termux as the command environment, Android hotspot/shared Wi-Fi as the simplest transport, Bluetooth as a fallback sharing path, Wi-Fi Direct as app/API-driven rather than automatic Termux mesh routing, and OpenPGP as the content protection layer.

Reference area	Public source
Termux app and package environment	F-Droid Termux page — f-droid.org/en/packages/com.termux/
Termux package management	Termux packages wiki — github.com/termux/termux-packages/wiki/Package-Management
Termux:API	Termux:API GitHub README — github.com/termux/termux-api
Android Wi-Fi Direct / P2P	Android Developers — developer.android.com/develop/connectivity/wifi/wifi-direct
GnuPG commands and key management	GnuPG official manual — gnupg.org/documentation/manuals/gnupg/
QR encoding (qrencode)	qrencode / libqrencode — fukuchi.org/works/qrencode/